

Programación I / Algoritmos y Estructuras I

Orientador de Clase

Ruptura de ciclos y manejo de excepciones

Temario: finalización anticipada de ciclos con break, ciclos controlados con while True, concepto de excepción, cláusulas try-except-else-finally, captura de excepciones específicas e instrucciones raise y assert.

1. Finalización anticipada de un ciclo: break

La instrucción break finaliza inmediatamente el ciclo for o while más cercano. La ejecución continúa en la primera instrucción ubicada después de ese ciclo. No finaliza por sí sola la función ni el programa completo.

Implementen es_primo(numero) utilizando una variable lógica y break. La función deberá retornar False para todo número menor que 2 y detener la búsqueda apenas encuentre un divisor.

- Prueben con -3, 0, 1, 2, 9, 17 y 25.
- Indiquen en qué caso se ejecuta break.
- Expliquen por qué puede evitar iteraciones innecesarias.
- Comparen el resultado con una versión que recorra todos los posibles divisores.

2. Ciclos controlados con while True

while True crea un ciclo cuya condición siempre es verdadera. Debe utilizarse solamente cuando la condición de finalización se comprueba dentro del bloque y conduce de manera clara a un break.

Desarrollen un programa que solicite números enteros y finalice cuando se ingrese -1. El valor -1 es la condición a controlar: controla la finalización y no debe incorporarse a los cálculos.

Al finalizar, informar cantidad, suma y promedio. Si el primer valor ingresado es -1, mostrar que no existen datos para calcular el promedio. No utilicen while True para reemplazar un for cuando la cantidad de repeticiones ya se conoce.

3. Concepto de excepción

Una excepción es una situación anormal detectada durante la ejecución del programa. Si no se captura, interrumpe el flujo normal y Python muestra un traceback con el tipo de excepción, el mensaje y la línea en la que se produjo.

```
numero = int(input("Ingrese un número: "))
resultado = 100 / numero
print(resultado)
```

Ejecuten el programa con 10, 0 y "hola". Identifiquen qué casos generan ZeroDivisionError y ValueError. Diferencien además una validación preventiva con if de la captura correctiva mediante try-except.

4. Captura con try-except

El bloque try contiene únicamente las instrucciones que pueden generar el error esperado. Cada except debe capturar una excepción específica. Eviten except sin tipo, porque oculta la causa del problema y dificulta la depuración.

Modifiquen el ejercicio del punto 3 para capturar ValueError cuando el usuario ingrese un dato no entero. El programa deberá informar el error y continuar solicitando valores sin perder los datos válidos ya cargados.

5. Múltiples excepciones y reglas del problema

```
numeros = [10, 20, 30, 40]
posicion = int(input("Posición: "))
print(numeros[posicion])
```

Capturen ValueError cuando el dato no pueda convertirse a entero e IndexError cuando la posición quede fuera del rango de la lista. Recuerden que los índices negativos son válidos en Python; si la regla del problema solo permite posiciones entre 0 y len(numeros)-1, deberán validarlo expresamente y provocar un ValueError.

Prueben con "dos", 1, 8 y -1. Expliquen por qué conviene mostrar un mensaje diferente para cada problema.

6. Cláusulas else y finally

Desarrollen un programa que solicite dos números y calcule su división. Utilicen:

- except ValueError para datos no numéricos.
- except ZeroDivisionError para un divisor igual a cero.
- else para mostrar el resultado solamente si el bloque try finalizó sin excepciones.
- finally para mostrar un mensaje de cierre que se ejecute exista o no un error.

El bloque else no reemplaza al try y finally no indica que la operación fue exitosa: solamente garantiza la ejecución de tareas finales, como cerrar un recurso o informar que el intento terminó.

7. Provocar una excepción con raise

raise permite señalar que un dato no cumple las reglas del problema, aun cuando su tipo sea correcto. Implementen calcular_importe(cantidad, precio) de modo que provoque ValueError si cantidad o precio son menores o iguales a cero. La función deberá retornar cantidad * precio cuando los datos sean válidos.

Desde el programa principal, capturen la excepción y muestren su mensaje utilizando except ValueError as error. Prueben con datos válidos, cantidad cero, precio negativo y entradas no numéricas.

8. Comprobaciones internas con assert

assert comprueba una condición que el programador espera que siempre sea verdadera. Si resulta falsa, genera AssertionError. Es una ayuda para detectar errores de lógica o verificar invariantes internas; no debe utilizarse como mecanismo principal para validar datos ingresados por el usuario, ya que las aserciones pueden desactivarse.

```
importe = calcular_importe(3, 1200)
assert importe == 3600, "El importe calculado no es el esperado"
```

Escriban otras dos aserciones: una que verifique un caso correcto y otra que falle intencionalmente. Interpreten el mensaje de AssertionError obtenido.

9. Desafío integrador: carga segura de productos

Desarrollen un sistema modular que registre productos en una lista. Cada producto podrá almacenarse como una lista con código, descripción, cantidad y precio. La carga finaliza cuando el código ingresado sea "FIN".

El programa deberá:

- Utilizar while True y break para finalizar la carga.
- Capturar ValueError al convertir cantidad y precio.
- Utilizar raise ValueError para rechazar cantidades o precios menores o iguales a cero.
- No incorporar un producto cuando sus datos sean inválidos.
- Calcular el importe de cada producto mediante una función.
- Utilizar assert para comprobar una condición interna del cálculo, no para validar la entrada.
- Informar cantidad de productos, importe total y precio promedio; si no se cargaron productos, evitar la división por cero.

10. Verificación y análisis

Prueben el desafío con los siguientes casos y registren resultado esperado y obtenido:

- a) Finalización inmediata con FIN.
- b) Producto válido.
- c) Cantidad escrita con letras.
- d) Precio igual a cero.
- e) Varios productos válidos antes de FIN.
- f) Una aserción que falle de manera intencional durante la depuración.

Actividad para el Portfolio

Entregar un único documento grupal que contenga:

- Código fuente de todos los ejercicios y del desafío integrador.
- Capturas de ejecución de casos válidos y de cada excepción trabajada.
- Explicación de la diferencia entre break, try-except, raise y assert.
- Justificación del uso de else y finally en el ejercicio de división.
- Tabla de casos de prueba con entrada, resultado esperado y resultado obtenido.
- Reflexión individual: ¿qué estrategia les resultó más útil para construir programas robustos y por qué?